

```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
    27.9/27.9 MB 45.4 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
import gensim
print("Gensim installed successfully")
```

Gensim installed successfully

STEP 1— Import Libraries

```
import gensim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

STEP 2 — Load Pre-trained Word2Vec

```
import gensim.downloader as api

model = api.load("glove-wiki-gigaword-50")
```

[=====] 100.0% 66.0/66.0MB downloaded

STEP 3 (Continue) — Explore Model

```
print("Vocabulary size:", len(model))
```

Vocabulary size: 400000

```
print("Vector dimension:", model.vector_size)
```

Vector dimension: 50

```
print(model['king'])
```

```
[ 0.50451  0.68607 -0.59517 -0.022801  0.60046 -0.13498 -0.08813
 0.47377 -0.61798 -0.31012 -0.076666  1.493  -0.034189 -0.98173
 0.68229  0.81722 -0.51874 -0.31503 -0.55809  0.66421  0.1961
-0.13495 -0.11476 -0.30344  0.41177 -2.223  -1.0756 -1.0783
-0.34354  0.33505  1.9927  -0.04234 -0.64319  0.71125  0.49159
 0.16754  0.34344 -0.25663 -0.8523  0.1661  0.40102  1.1685
-1.0137  -0.21585 -0.15155  0.78321 -0.91241 -1.6106 -0.64426
-0.51042 ]
```

STEP 4 — Word Similarity (10 Pairs)

```
pairs = [
    ('doctor', 'nurse'),
    ('cat', 'dog'),
    ('car', 'bus'),
    ('king', 'queen'),
    ('man', 'woman'),
    ('apple', 'orange'),
    ('school', 'teacher'),
    ('river', 'water'),
    ('city', 'village'),
    ('computer', 'internet')
```

```
]

for w1, w2 in pairs:
    print(f"{w1} - {w2}: {model.similarity(w1, w2):.4f}")
```

```
doctor - nurse: 0.7977
cat - dog: 0.9218
car - bus: 0.8211
king - queen: 0.7839
man - woman: 0.8860
apple - orange: 0.5388
school - teacher: 0.7855
river - water: 0.6191
city - village: 0.7453
computer - internet: 0.8060
```

STEP 5 — Find Top Similar Words

```
words = ['king', 'university', 'doctor', 'india', 'music']

for word in words:
    print("\nTop similar words to:", word)
    for similar_word, score in model.most_similar(word, topn=5):
        print(f"{similar_word} ({score:.4f})")
```

```
Top similar words to: king
prince (0.8236)
queen (0.7839)
ii (0.7746)
emperor (0.7736)
son (0.7667)

Top similar words to: university
college (0.8745)
harvard (0.8711)
yale (0.8567)
graduate (0.8553)
institute (0.8484)

Top similar words to: doctor
nurse (0.7977)
physician (0.7965)
patient (0.7612)
child (0.7559)
teacher (0.7538)

Top similar words to: india
indian (0.8649)
pakistan (0.8530)
malaysia (0.8167)
bangladesh (0.8154)
delhi (0.8143)

Top similar words to: music
musical (0.8854)
pop (0.8682)
dance (0.8531)
songs (0.8526)
recording (0.8392)
```

STEP 6 — Word Analogy Tasks

```
print("king - man + woman:")
print(model.most_similar(positive=['king', 'woman'], negative=['man']))

print("\nparis - france + india:")
print(model.most_similar(positive=['paris', 'india'], negative=['france']))

print("\nteacher - school + hospital:")
print(model.most_similar(positive=['teacher', 'hospital'], negative=['school']))
```

```
king - man + woman:
[('queen', 0.8523604273796082), ('throne', 0.7664334177970886), ('prince', 0.7592144012451172), ('daughter', 0.7473883628845), ('son', 0.7473883628845), ('emperor', 0.7473883628845), ('ii', 0.7473883628845), ('son', 0.7473883628845), ('emperor', 0.7473883628845), ('ii', 0.7473883628845)]

paris - france + india:
[('delhi', 0.8888585567474365), ('bombay', 0.8200730085372925), ('mumbai', 0.805435061454773), ('lahore', 0.770347237586975), ('bangladesh', 0.770347237586975), ('pakistan', 0.770347237586975), ('malaysia', 0.770347237586975), ('india', 0.770347237586975), ('delhi', 0.8888585567474365), ('bombay', 0.8200730085372925)]

teacher - school + hospital:
[('nurse', 0.8027187585830688), ('doctor', 0.7993789911270142), ('hospitalized', 0.7383416295051575), ('patient', 0.72919525), ('physician', 0.72919525), ('school', 0.72919525), ('hospital', 0.72919525), ('nurse', 0.8027187585830688)]
```

STEP 7 — Visualize Word Embeddings Using PCA

```
words = [
    'king', 'queen', 'man', 'woman', 'prince', 'princess',
    'doctor', 'nurse', 'hospital', 'teacher', 'school',
    'paris', 'france', 'india', 'delhi', 'pakistan',
    'car', 'bus', 'train', 'music', 'dance', 'college'
]
```

```
vectors = [model[word] for word in words]
```

```
from sklearn.decomposition import PCA

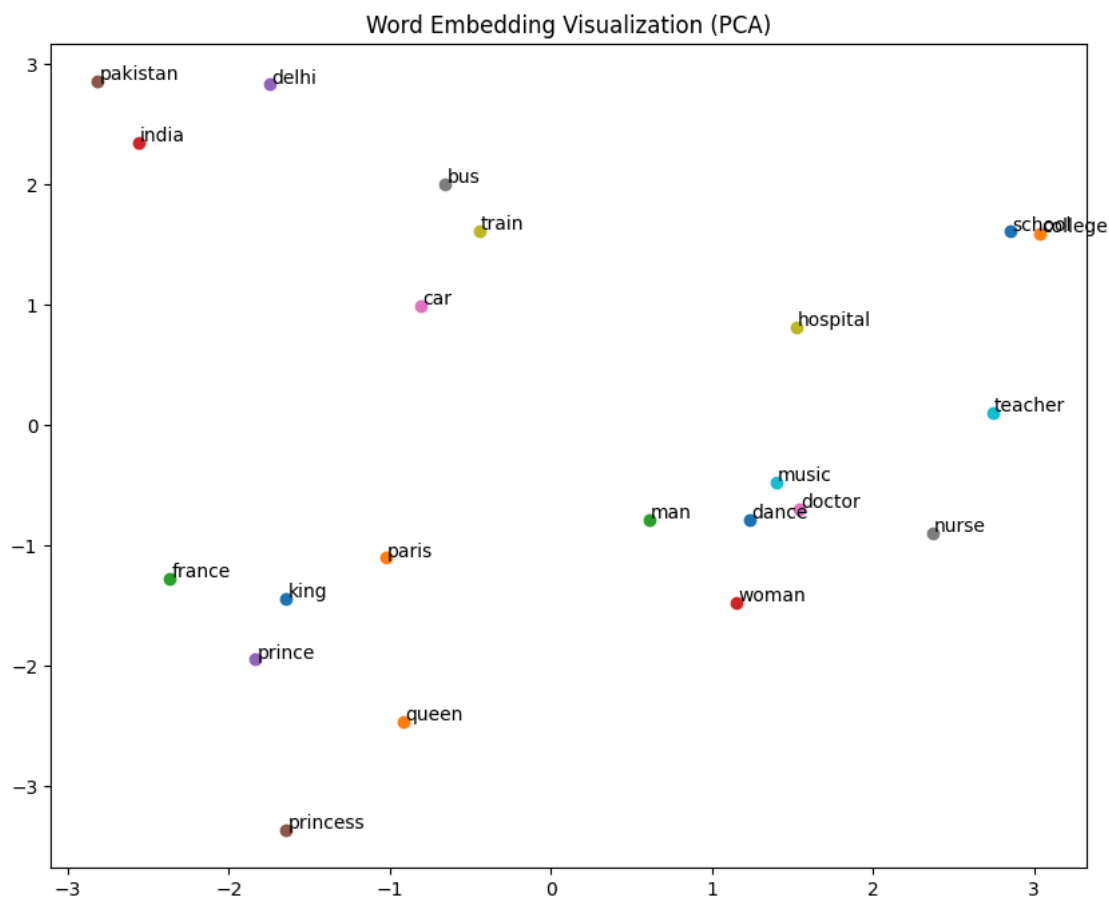
pca = PCA(n_components=2)
result = pca.fit_transform(vectors)
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))

for i, word in enumerate(words):
    plt.scatter(result[i,0], result[i,1])
    plt.text(result[i,0]+0.01, result[i,1]+0.01, word)

plt.title("Word Embedding Visualization (PCA)")
plt.show()
```



STEP 8 — Reflection and Interpretation.

Word embeddings represent words as dense numerical vectors that capture semantic meaning. Unlike one-hot vectors, embeddings store contextual relationships between words in continuous vector space. In this experiment, similar words such as “king” and “queen” or “doctor” and “nurse” were located close to each other, showing that the model learned meaningful relationships from large text corpora. The similarity scores demonstrated that semantically related words have higher cosine similarity values.

The analogy tasks such as king – man + woman $\approx$ queen show that embeddings capture relational patterns like gender and geography through vector arithmetic. This happens because relationships are encoded as consistent vector directions in the embedding space.

However, embeddings may sometimes reflect biases present in training data, and some analogy results may not always be perfectly accurate. Overall, embeddings effectively capture semantic similarity and contextual information, making them powerful for NLP applications such as chatbots, search engines, and translation system